

Model-based construction of collaborative systems

M A Swaby, P M Dew and P J Kearney

The principal asset of an enterprise is its workforce, and a key factor distinguishing the successful enterprise of the forthcoming millennium is the empowerment of its staff in pursuit of corporate goals. While the more routine tasks will become totally automated, there will also be an increased use of IT to amplify the abilities of knowledge workers as individuals and members of collaborative teams. These teams are likely to be much more volatile than at present, being convened for specific purposes and disbanding once the purpose is achieved. Individuals may simultaneously belong to several teams, and may play different roles in each of these. This view of the future poses challenging requirements of the supporting software infrastructure. Many support applications for virtual teams share similar fundamental requirements for core services, such as information management, co-ordination and collaborative working. This paper proposes an approach to construction of collaborative systems in which a model is used to describe user interaction with core services within an application context. This model then drives a run-time enactment service which provides integrated access to required services through appropriate user interfaces. This development approach enables flexible integration and customisation of core services for team members, and enhances the ability of the software to evolve over time as requirements change. An experimental model-based collaborative system was built according to this approach, and applied to a virtual team scenario based on a BT business process for provision of quotations for network services. Evaluation trials were encouraging and demonstrated both the feasibility and usability of the approach, particularly for participatory design of collaborative systems with end users.

1. Introduction

In the future, the most successful enterprises are likely to be those which can manage and evolve their activities within increasingly dynamic business environments. While more routine activities will become totally automated, there will also be an increased use of information and communications technologies to amplify the abilities of knowledge workers as individuals and as members of virtual teams [1]. These teams bring together distributed groups of co-operating individuals who contribute complementary skills towards a common activity. The teams are likely to be much more volatile than at present, being convened rapidly for specific purposes and disbanding once that purpose is achieved. Individuals may simultaneously belong to several teams, and may play different roles in each of these.

Recent advances in enterprise computing technologies such as intranet and middleware architectures are key enablers for virtual teams. These technologies are already widely used for delivering distributed information and collaborative services across enterprise networks. Hence, they are highly appropriate in supporting the integration of distributed participants, information and resources to support a virtual team. However, in addition to overcoming their distributed nature, the next generation of software support infrastructure must also address two further characteristics that are currently less well supported:

- heterogeneity — it must support a broad range of individual and collaborative tasks performed by different people, with differing roles, responsibilities and requirements,
- dynamism — it must dynamically adapt and evolve to reflect more rapid changes in the business process environment.

This paper describes a model-based approach to construction of collaborative software systems which reflects the heterogeneous and dynamic characteristics of virtual teams [2]. Specifically, a model-based architecture is proposed which provides an abstraction of the interaction between users and the supporting software services within an application context. There are two related objectives in the approach. Firstly, to improve the ability of software engineers to rapidly develop customised collaborative applications that are tailored to the heterogeneous requirements of virtual teams. Secondly, to improve the ability of these applications to dynamically evolve over time as requirements change. The paper firstly considers the nature of model-based collaborative software systems appropriate to supporting virtual teams (section 2). Secondly, the proposed model-based approach is described (section 3). The approach was evaluated using a realistic demonstrator applied to a simplified BT business process

for quotation of network services (sections 4, 5 and 6). Finally, results of the evaluation are presented and conclusions drawn (sections 7 and 8).

2. Model-based software to support virtual teams

Virtual team working is typically characterised by the following attributes:

- geographical separation of team members,
- custom process models,
- high-value 'knowledge workers' interleaving work on several projects, probably using different process models on each,
- integration of individuals with different personal work-styles, technical specialisms, and quite possibly different corporate cultures.

Not only does this require the rapid provision of the necessary IT support (access to data resources, communication facilities, application software, etc), but also a high degree of flexible customisation. Generic process models will have to be adapted to the specific project requirements and team make-up. Also, from an individual worker's point of view, their working environment must adapt to the requirements of the current task without losing its integrity as a personal workspace. Conventional monolithic workflow management systems provide robust support for business process co-ordination, but are not ideally suited to supporting virtual teams where less prescription and more flexibility is often required. Hence, there is considerable interest in holistic approaches to workflow management in which traditionally 'co-ordination-centric' workflow concepts are converging with traditionally 'collaboration-centric' groupware concepts from the computer support for co-operative work (CSCW) community.

An important notion within these holistic approaches to CSCW is the architectural separation of the required software services into a number of orthogonal classes (see, for comparison, the classifications described elsewhere [3—6]). Sheth, for example, envisages that next generation workflow management systems will integrate co-ordination, collaborative working and information management services to create flexible work co-ordination and collaboration systems (WCCS) [6]. Co-ordination services manage the interdependencies between people, tasks, information and resources within a business process. Collaboration services provide support for human/human interaction, e.g. audiovisual conferencing, application sharing and whiteboarding. Information management services integrate and unify required information sources into a virtual information space appropriate to the work context. In constructing a particular software application within such environments, it is desirable to focus software

development effort towards integration of these reusable services (i.e. co-ordination, collaboration and information management services), rather than develop applications from scratch. In fact, most existing implementations of these services (referred to in this paper as 'core' services) are designed explicitly for reuse through their model-based architectures, which separate logical specifications from a general-purpose enactment service driven by those specifications. For example, workflow management systems typically separate business process description from process enactment; database applications separate the entity relationship model from the database engine, etc. Within these model-based core service implementations, the use of an abstract model simplifies an otherwise complex system. This enhances the ability of the system to evolve because adaptation can be affected at an abstract level (a detailed discussion of enterprise integration through models can be found in Petrie [7]).

Similar properties are also desirable in the collaborative applications that may be composed from these core services. Middleware architectures such as CORBA [8] can facilitate basic software level interoperability, but there is a requirement for higher level modelling abstractions that capture how services are composed together and utilised as integrated applications by end users. It is this requirement which is addressed here. A modelling approach, together with an associated software architecture, has been developed to flexibly support the heterogeneous and dynamic characteristics of virtual teams.

3. User interaction modelling for collaborative systems

In creating a dynamic software environment to support the activities of a heterogeneous virtual team, it is desirable to integrate reusable implementations of core services where possible. Application specific functionality may then be layered above these reusable services to create a tailored application with good maintenance and evolution qualities. Given the advantages and availability of model-based implementations of core services, an integrated model-based approach to the construction of whole applications has been explored, composed from these services. The approach is to model the user interaction with core services within a collaborative application context, as shown conceptually in Fig 1.

The approach shown in Fig 1 is oriented towards two main software development issues with respect to integrated collaborative applications:

- functionality — what core services are required and how they are composed together to create a collaborative application,
- user interaction — how users interact with applications composed from these core services.

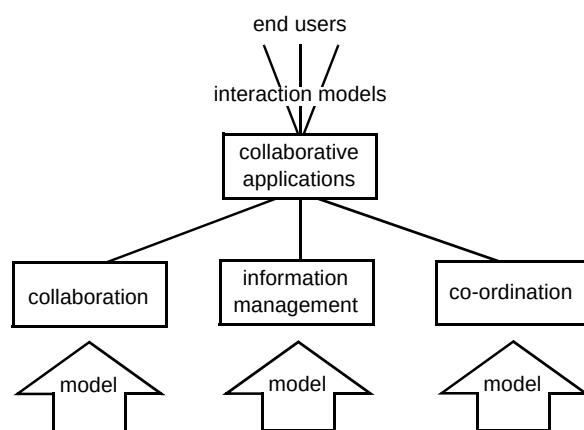


Fig 1 User interaction modelling for collaborative applications.

This approach uses an object-oriented model to integrate application functionality with the embodiment of that functionality at the human/computer interface. This model may then serve as a run-time specification for an application 'enactment service' which can dynamically provide users with access to the services they require via an appropriate user interface.

At the modelling level, the requirement is to capture user interaction with the functional services from which a collaborative application is comprised in a general way. In achieving this, an existing object-oriented scripting language called DMSL (display metaphor scripting language) was adopted [9]. DMSL was initially designed to support automatic generation of Web user interfaces for navigation of large-scale information spaces [10]. In this work, DMSL was extended to enable specification of a general model of user interaction with collaborative applications. DMSL captures user interaction with collaborative applications according to the framework described as follows.

Users of the collaborative application interact with the software environment via a compound user interface comprised of interactor and descriptor objects. An interactor is an interface component which handles user input events (e.g. a text box or menu). A descriptor is a presentational representation for an application or information object (e.g. a bar chart representing a set of numbers). Interactors and descriptors may be composed together within containers, which may then themselves be regarded as components. Components define properties through which their state may be set or retrieved and which specify actions to take when particular events occur. Actions received through interactors trigger methods within the collaborative application or the core services from which it is comprised.

The DMSL scripting language is designed to describe user interaction independently of any particular object-oriented user-interface paradigm. This independence is

central to the model-based approach in supporting a heterogeneous range of user interactional requirements. That is, an interaction model expressed in DMSL may be implemented through several different concrete user interfaces.

A user interaction model for a collaborative application adhering to the above framework is created within DMSL using language constructs called DMSL components. Components are used to represent interactors and/or descriptors within the application user interface. Core definitions may be subclassed and composed together to create more complex user-interface definitions. The simplified grammar of a DMSL component is shown below. (Line numbers are for reference only, and are not part of the language.)

```

1 define <id> extends <id>
2   used_to_display <object class>
3   within <container>
4   in_style <style>
5 {
6   [<variable slots>]
7   [<content and operator slots>]
8 }
  
```

Line 1 provides a unique name for the interface component object, and (optionally) the name of a direct superclass from which it inherits. Line 2 associates the component with a type of object within the application environment for which it is designed to provide a user interface representation. Where a component may be used within a number of different containing components, the name of a specific container may be stated (line 3). Finally, an arbitrary string token may be used to group together components into particular sets, which together create a particular look and feel or interface metaphor (line 4). A number of local variables may be defined using integer, floating point or string types (line 6). Content slots specify the user-interface components from which a particular container interactor is composed (line 7). Operator slots specify actions to perform when a particular run-time event occurs through the component.

The structure of content and operator slots is shown below.

```

1 add <id>[(parameters)]
2   [at <location>]
3   [when <condition>]
4   [action <event type> <action>]
  
```

Line 1 adds a previously defined DMSL component into the current container, optionally initialising it with parameters which override default values of local variables in the target component. Where the container supports object layout (e.g. <x, y> positioning of objects within a panel), a relative position can optionally be specified (line

2). If no such indication is provided, the software implementation is responsible for object layout. Addition of a component to the container can be conditional upon run-time evaluation of a Boolean expression (line 3), e.g. the value of a local variable.

Finally, a number of actions may be associated with particular run-time events which occur through an interactor, such as mouse selection or keyboard input (line 4). Typically, actions invoke operations upon core services implemented within a distributed object architecture. Within this work, the CORBA [8] architecture and associated services [11] were used during implementation. However, object requests are modelled in DMSL using a simplified representation suitable for implementation using other distributed object technologies (e.g. DCOM, Java RMI, etc). The syntax of a DMSL object request is shown below.

```
call <obj>.<method>(<params>)
```

where <obj> is the name of a unique run-time service in the distributed object environment and <method> identifies a particular method within the service that accepts the input parameters specified in <params>. The purpose of this construct is not to duplicate or replace an interface definition language (IDL) [8] or object request scripting language, e.g. CorbaWeb [12] within the distributed computing environment. Rather, it is used to direct the values of user interface component properties through to services, which operate upon them following interface events.

The simplest DMSL component definitions map directly on to basic container and component objects within concrete user interface application programming interfaces (APIs). In Web-based implementations for example, base DMSL components would represent atomic page elements such as frames, paragraphs, in-line images, etc. By composing basic definitions together within the content slots of new container definitions, more complex interface components can be specified. A key object-oriented feature of DMSL is support for interface component inheritance through the 'extends' syntax introduced earlier. By default, all slots within a parent component are inherited by a child subclassed from it, i.e. variable, content and operator slots. Inheritance within DMSL allows new constructs to be added to variable and content slots, and a limited capability for slot overriding or removal is also provided. The syntax for achieving this is:

```
1  remove <id>
2  replace <id> with <component>
```

where line 1 removes the variable or content slot entry specified by <id> and line 2 replaces a variable or content slot entry with a new entry, specified using the 'add' syntax.

For DMSL to be useful in creating flexible and reusable components, the encapsulation and inheritance properties of the language should be exploited during component design. One objective in using an object-oriented modelling language is to enable creation of reusable component 'libraries' that may be quickly and easily drawn together and specialised to create new ones. These libraries are most useful when structured into a number of separate layers, each of which provides a well-defined foundation for new applications to be built, or for new groups of components to be added. While lower layers provide general-purpose reusable user-interface components, the highest layers are often customised to particular end-user roles within the collaborative application. These customised sets of DMSL components provide a high-level abstract model of how people performing a particular role interact with core services within the application.

An extract of a typical DMSL component is shown below. This component is based upon a Web-based VRML interaction style and specifies a 3-D office composed from objects such as desks, chairs, etc. Objects within the room are interactive (e.g. clicking on the telephone invokes an audio call).

```
1  define Office
2  extends Room
3  used_to_display Person
4  in_style 3D
5  {
6    set desk to_contain {
7      add phone1 as Telephone() at 1,1
8      action select conf.audiocall($user);
9    }
10 }
```

This DMSL component is associated with a 'Person' object within the information space for the application. The \$user token in line 8 refers to an attribute of the 'Person' class. Through this association, the component may be used to generate an interactive 'virtual office' for any user described by an instance of the 'Person' object class. A subclass of the component can be created to customise the office for a particular role or individual. For example:

```
1  define NetworkDesignOffice
2  extends Office
3  used_to_display Person
4  in_style NetworkDesign3D
5  {
6    set desk to_contain {
7      add pc1 as Computer() at 1,2
8      action select tool.login($user);
9    }
10 }
```

Here the DMSL component is customised for a 'network designer' role. A computer object is added to the

desk to enable access to a network design tool. All other DMSL components and variables are automatically inherited from the parent 'Office' DMSL component (e.g. the desk and phone objects).

4. A model-based user interaction service

The interaction model for a collaborative application is defined through a set of DMSL components. These components are then used to control a run-time enactment service, called the 'model-based user interaction service.' The service is responsible for providing users with appropriate user interfaces supporting integrated access to required core services as defined in the DMSL interaction model. The architecture of the service, showing the linkage between 'build time' when models are defined and 'run time' when they are enacted, is shown in Fig 2.

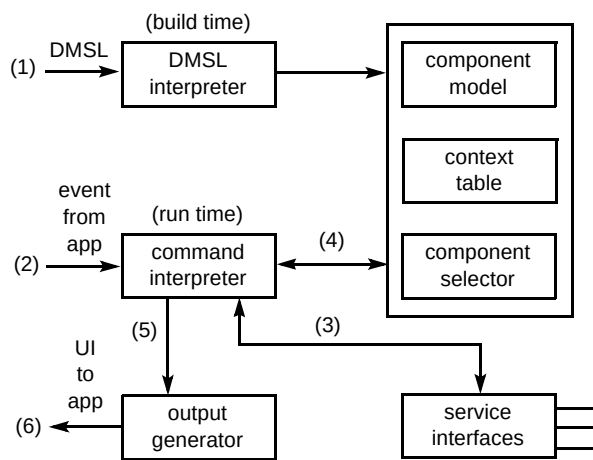


Fig 2 Model-based user interaction service.

In the build-stage, models are loaded into the service via a DMSL interpreter (step (1) in Fig 2). Because of the object-oriented nature of the DMSL language, component definitions are loaded in order of inheritance, i.e. basic component definitions are loaded prior to the components derived from them. The DMSL interpretation process builds up two in-memory representations of the interaction model. Firstly, each DMSL component is represented within an internal 'component model'. The component model represents the composition and inheritance of component definitions using a tree structure. Secondly, contextual information associated with definitions through 'used to display' or 'style' constructs is used to build up a tabular index. This index enables an appropriate interface component to be selected dynamically in response to a run-time request from an application. Once the set of DMSL components for an application has been interpreted, the user interaction model is ready for enactment.

At run time, the model-based user interaction service (Fig 2) is driven primarily by a single DMSL language construct called the 'show' command. The 'show' command instructs the user interaction service to select and

generate a (composite) interface component, based on one of three strategies:

- generation of a specific named component,
- automatic selection of a component within a specific interaction context,
- automatic selection of component and context.

The simplest form of the 'show' command directs the user interaction service to generate a particular named interface component. The DMSL form of the command is shown below:

```
show <component>(parameters)
```

For example, the following command would generate a message box containing the text 'hello world':

```
show MessageBox(text='hello world')
```

The two more powerful forms of the 'show' command do not specify a particular component to generate. Rather, they rely upon the user interaction service to automatically select an appropriate user interface for one or more 'objects' within the application. These objects represent the results of executing methods within the collaborative application, or the core services from which it is composed. The structure of the most general form of the show command is:

```
show <service>.<method>(params)
```

for example:

```
show security.login(uid=john, password=xxxx)
show info.find(query="select * from t1")
```

As discussed earlier, a simplified approach to handling remote method invocations, and their results, is taken within the user interaction service. A number of steps are taken by the user interaction service in processing this type of command, typically received via a user interface event (step (2) in Fig 2). Firstly, the specified service is located and the relevant method is invoked with parameters where supplied (step (3)). Remote methods must return a set of one or more result objects which are of a simplified form suitable for processing by DMSL components. A result object is defined as a set of name-value properties (i.e. fields or attributes). One property within the result object must specify the 'class' of the object. Following remote method invocation, the command interpreter attempts to locate a user interface component for each element of the result set (step (4)). It achieves this by consulting the context table and attempting to match the class of a result object with that associated with an interface component.

For example, a log-in operation might always return a single object of class 'LoginResult', containing a string property named 'Status' with a value of 'welcome' or 'access denied'. At run time, a component associated with this class would access the 'Status' property as a local variable, and would be able to represent its value appropriately (e.g. as a text entry within an HTML frame), for example:

```
define loginhandler
used_to_display LoginResult
in_style WebStatusBar
{
    add text1 as HTMLText(text=$Status)
}
```

where WebStatusBar is the name given to this style. The resultant sets returned by remote methods within the application or core services are not required to be of a single class. For example, a database query might return a set of objects of several classes. In this scenario, the component selector attempts to match each object in the result set individually. The command interpreter is then responsible for the layout of components within an appropriate container (e.g. representation of a query result set as rows in a table). If the application has been designed and built correctly, it should always be possible to match at least one component to any result object. However, if the component selector finds no matches, it generates a predefined 'error' component containing information about the result object and the command which returned it.

In applications that support multiple interaction styles (e.g. Web applications providing integrated HTML and VRML interaction), the 'show' command may be contextualised using a 'style' parameter. For example, an object of class 'Organisation' might be represented in 'structure chart' or 'employee list' styles. Styles have two major uses — firstly, they modularise particular interaction styles (e.g. 2D HTML, 3D VRML), and, secondly, they provide different views on objects within the application. Styles can, for example, be used to create dynamic customised views of the application for each class of user while utilising the same core services.

When the 'show' command has been interpreted and appropriate components have been selected, they are passed to an output generator as a tree (step (5) Fig 2.) The tree structure passed to the output generator reflects the hierarchical composition of compound user interfaces from containers and components. The output generator is then responsible for creating the target user interface by traversing the tree and 'applying' each node within it. This mechanism is adopted from the scene-graph approach used in 3-D interface construction [13]. During the generation process, DMSL 'show' commands for the next stage of user interaction are embedded within the user interface. The interface is then made available to the user and the event processing cycle continues (step (6)).

5. Implementation of a model-based collaborative system (MCS)

The user interaction service is the central component of a distributed system architecture for collaborative applications in which core services are flexibly integrated and delivered according to models. These systems are referred to as model-based collaborative systems (MCS). This section describes a prototype MCS implementation, built upon Web and CORBA infrastructure, as shown in Fig 3. The prototype enables exploration of integrated (2-D) HTML and (3-D) VRML user interfaces to collaborative applications comprised of CORBA-based co-ordination, information management and collaborative working services.

Translation of Web browser user interface events to object requests within the CORBA environment is supported through an 'ORB bridge.' This is implemented as a common gateway interface (CGI) process running within a dedicated Web server. At the browser, DMSL 'show' commands are embedded within Web interface elements as parameters of hyperlink URLs directed to the CGI/ORB bridge. The bridge reformulates these parameters as a CORBA object request, which is invoked upon a 'controller object' for the collaborative application.

The controller object implements application-specific logic and facilitates integration of core services independently of DMSL, if required. DMSL describes high-level integration of core services as manifested visually at the user interface. However, applications will often need to integrate and access these services irrespective of user interface requirements. The controller object serves as a general-purpose architectural component where this logic can be implemented. A key purpose of the user interaction service is to significantly reduce the amount of specific logic that must be 'hard-coded' into the application layer, by replacing this code with DMSL components where appropriate.

Upon completion of event processing within the controller (or immediately if no processing is necessary), the embedded DMSL 'show' command is redirected to the user interaction service.

The user interaction service was built using tools from the University of Leeds DiMe system, built by Morris and Lajos [9]. Within DiMe, a component definition and command interpreter for DMSL was implemented in C++ using the PCCTS compiler construction tool-kit [14].

Within the MCS prototype, the DMSL interpreter was encapsulated as a CORBA object accessed via an IDL interface. A multi-threaded server was developed to mediate parallel access to interpreter objects on a per-user basis.

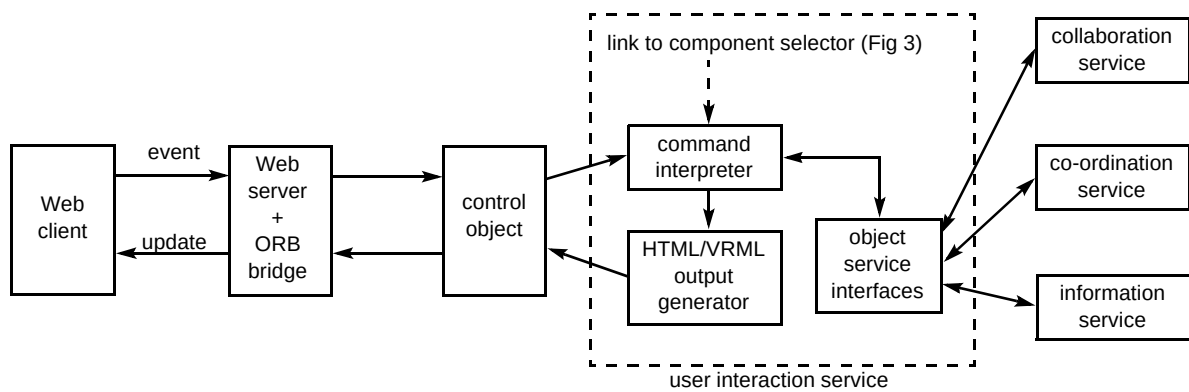


Fig 3 Implementation of the user interaction service within a model-based collaborative system (MCS).

As the 'show' command is interpreted within the user interaction service, DMSL components are selected and composed together. Where a DMSL component references a core service, a CORBA object request is constructed and its parameters instantiated. The service is then invoked via 'object services interfaces,' which locate and route requests to appropriate servants within the CORBA environment. Three prototypical core service implementations were developed, as described below.

- Co-ordination

A co-ordination service was provided through development of a skeletal workflow enactment engine, based informally upon interfaces defined within the WFMC reference architecture [15]. The process interchange format (PIF) language [16] was used to represent both business process structure and instance information. State changes in the co-ordination service over time were simulated through the submission of a series of models into the service via a PIF parser.

- Information

An information service was built as a Java implementation of the CORBA queryable collection service [11]. In the prototype, the information model and information object instances were loaded in a textual format (via a parser for data in LDIF format — lightweight directory access protocol data interchange format [17]).

- Collaboration

A simple collaboration service was constructed, supporting person-to-person synchronous groupware tool execution via audiovisual conferencing and shared whiteboarding. A simple CORBA client was developed to support tool execution and session control at the client, independently of browser security restrictions. Model-level support for this service was provided through an extension of the PIF co-ordination model. The core PIF classes were extended (using the partially shared views (PSV) mechanism [18]) to

associate collaboration information with activities via a new relation. This relation was used to link collaborators to a common task, and to specify which tools to use (whiteboard, audio or video) in collaboration on the task.

Following completion of object requests upon core service implementations, the command interpreter passes the tree of generated DMSL components and result object sets to output generators for HTML and VRML. The HTML output generator component was built using an extended proprietary implementation of (what has now become) the W3C Document Object Model (DOM) [19]. 3-D interface generation was supported through VRML 1.0 [20] using the SGI Inventor libraries [13]. Following output language generation, streams are returned to the browser for rendering, passing back through the control object and CGI process within the Web server. 'Show' commands for the next cycle of user interaction are embedded as Web hyperlinks within the resultant user interface.

6. Case study

In order to evaluate the MCS prototype, its application towards supporting a virtual team within a BT business process was considered. In the following section, an overview of the business process used within the case study is given. The case study scenario for use of the MCS prototype within this process is then summarised, and the models used in its construction are described. A summary of feedback from the evaluation process is then presented.

6.1 Overview of the 'quotation for network services' business process

The case study used in the scenario was based upon a business process within BT through which quotations for data network services are provided to customers. As described in greater detail in Jennings [21], the quotation process requires co-ordination and collaboration between several departments within BT. A simplified view of the process is shown in Fig 4.

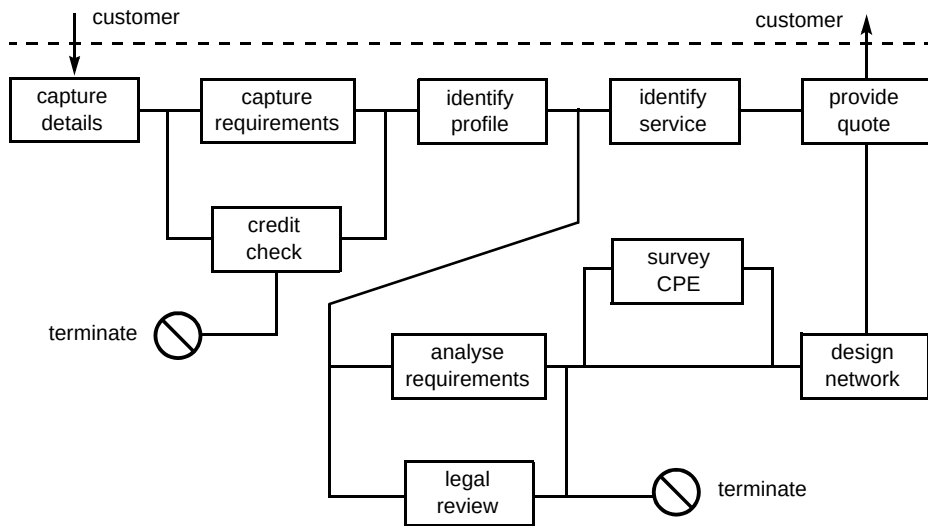


Fig 4 Provision of quotations for network services.

The process takes a customer enquiry for a network service as input, and provides a quotation for provision of that service as output. A job is initiated when a customer approaches BT for a quotation. After capturing customer details, an account manager is assigned to the job. During this stage of the process the account manager liaises with the customer and builds up a more detailed picture of their requirements. While this is being completed, the customer may be vetted by an external credit reference agency. The output of the requirements capture task is a profile of the service needed by the customer. If the profile requirements can be met from the existing portfolio then a standard quotation is provided and the process terminates.

If the customer requirements cannot be met from the existing portfolio, a bespoke data network design must be created in order to provide a customised solution. At this point, a network design team is assigned to the job and a more detailed network analysis and design is conducted. While this is under way, a parallel review is conducted by a legal team to ensure that the emerging design does not contravene relevant legislation. During the design stage, the account manager mediates between the customer and the various co-operating groups within the virtual team (e.g. network designers, legal advisors and technical support staff). If it is deemed necessary during the secondary requirements analysis phase, the customer premises equipment (CPE) is surveyed on-site by engineers to provide input into the detailed network design activity. Upon initiation of the detailed design task, the design team transfers the customer details and requirements records from a central workflow system to a (textual) design database controlled by an account manager. During the design activity the design team uses a range of heterogeneous information sources and tools. For example, a designer may require access to information on geographical site location, protocols, vendor equipment, design guidelines, network performance characteristics, etc.

Once the design team has completed the bespoke network design for the customer, a profile of that design is fed back into the main workflow management environment and a customised quotation prepared. The quotation is then delivered back to the customer and the process terminates.

Several distinct roles must co-operate in order to provide a customer quotation (e.g. customer handling centre operative, account manager, network designer and legal advisor). Employees performing these roles may be located within business units at several sites around the UK. Hence the need for the software to support the computer-mediated 'virtual team.'

Team members have different roles, skills, expertise and responsibilities. These differing perspectives upon the shared process are reflected in differing requirements for the supporting software infrastructure. For example, the network designer role requires access to a range of information sources and software tools; the account manager would benefit from collaboration facilities to enable communication with the remote staff working on a quotation job; and the legal advisor requires a specialised view of the emerging requirements profile to assess its conformance to relevant laws, etc.

6.2 Case study scenario

It is assumed that the core co-ordination, collaboration and information services have been modelled and the requirement is now to support the geographically separated teams that are needed to undertake various stages of the quotation process. Within the quotation process, there are several distinct roles which must co-operate in order to provide a customer quotation. Four exemplar roles have been identified — the customer handling centre operative, account manager, network designer and legal advisor. General-purpose reusable user-interface components and

specific interaction models for the process participants are to be specified using DMSL, and enacted using the MCS implementation.

Evaluation of the case study was undertaken in conjunction with a number of BT research engineers studying aspects of enterprise computing to support business processes. The model-based approach and the MCS prototype were presented to the group and a number of use cases were demonstrated. Feedback was obtained through a general discussion and follow-up interviews. The two main use cases considered within the case study were:

- exploratory prototyping of user interface requirements for a particular role,
- evolutionary change in business process characteristics.

6.3 DMSL component library for the case study

The starting point for building the DMSL component library for the case study was to construct a simple combined co-ordination, collaboration and information model for the quotation process. The information model, as shown in Fig 5, was based upon a subset of PIF core classes augmented with representations for work-lists and work-items.

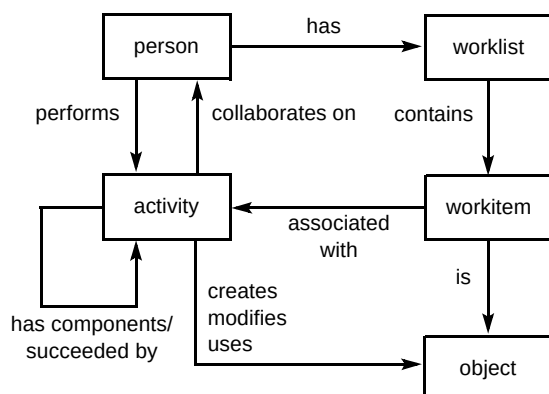


Fig 5 Co-ordination and information model.

The above model is used to describe relationships between people, tasks and information objects within the quotation process. A person interacts with the process support system via a work-list which contains a number of work-items which represent tasks for which they are responsible. In addition to being directly responsible for performance of a task (via the 'performs' relation), a person can also assist indirectly in the performance of a task (through the 'collaborates on' relation). The work-item representing a task can be associated with one or more information objects, e.g. the requirements capture task is associated with an information object in which customer requirements are characterised.

The DMSL component library for the case study was built up around the above model. Four basic types of interaction were considered:

- navigation through and presentation of the information objects used within the process,
- user interaction with the work-list either by the owner or supervisor,
- integration of collaborative working tools within the workflow client interfaces, e.g. shared whiteboarding and audiovisual conferencing,
- session control operations such as log-in/log-out and awareness of other users within the application.

A DMSL component library of interactors and descriptors supporting these types of interaction was built up hierarchically as shown in Fig 6. The DMSL component model was built upon a set of base primitives linking to HTML or VRML interface generation APIs. Base HTML components represent atomic containers and components (e.g. pages, frames, paragraphs, hyperlinks, in-line images).

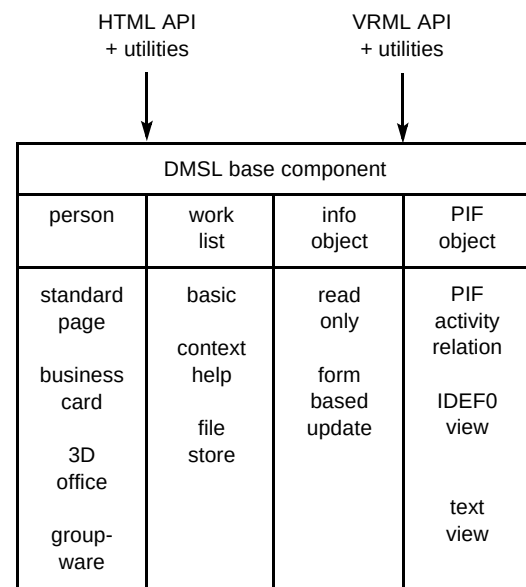


Fig 6 DMSL component library.

Utility components were then constructed over the base layer to provide general-purpose facilities such as form handling, presentation of information objects in tables, and access to the information service. The VRML base components enabled scene descriptors to be built up within the familiar 'room' metaphor used in the development of DiMe.

This included support for representations of 'physical' interactor DMSL components within 3-D scenes such as offices, desks, telephones, bookcases, and filing cabinets.

This allowed us to explore user interaction within the application in a style contrasting to HTML.

This base component layer consisted of a total of approximately 300 components, split between HTML and VRML components. For the quotation process case study, a single DMSL base component was created to define a generic look and feel for the application. This component defined around 30 variables defining standard colours, fonts, layouts, and other conventions for the application. This provided a similar functionality to style sheets for HTML interfaces, but covered both 2-D and 3-D interfaces. All subsequent components were then subclassed from the base component, thereby enabling the general look and feel of the entire application to be controlled from one place in the model.

The core of the interaction model was grouped into four sections. These sections consisted of a further 120 DMSL components, modelling user interaction with objects representing people, work-lists, work-items and process descriptions (Fig 6).

For example, for the person object (i.e. an entity containing person-oriented details such as name, address,

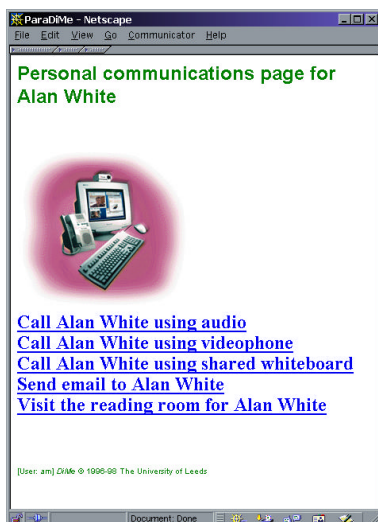
etc) a number of DMSL components were created to reflect different views and interaction context:

- a standard Web 'home page' view,
- a brief 'business card' view,
- a '3-D office' representation (VRML scene),
- a collaborative working view providing access to a collaborative working session via the Web page or the 3-D office component.

The DMSL work-list component enables the work-list to be presented in a tabular style as shown in Fig 7. Access to the work task owner 'A. White' is obtained by activating the link in the initiator column. Depending on company policy or the preferred interaction style of the user, a collaborative session can then dynamically be initiated through an HTML or 3-D office style, as also shown in Fig 7.

The final stage in construction of the interaction model for the quotation process was to define customised DMSL components, which tailor the general-purpose components described above for particular roles in the virtual team. For

Job id	Activity	Initiator	Date	Time	Status	Operations
1001	Provide Customer Quote (Customer Details)	A White	29-03-98	19:40	CustDetails	[Perform]
1002	Provide Customer Quote (Customer Requirements)	A White	03-05-98	14:00	ReqCap	[Perform]
1003	NEW Provide Quote (Network Design)	A White	04-05-98	09:34	NetDes	[Perform]
1000	Provide Customer Quote	C Smith	12-06-98	12:34	started	[Execute] [Exception] [Help]



Groupware tool execution in an HTML interaction style



Groupware tool execution in a '3D office' interaction style

Fig 7 Display of work-list and alternative styles of the 'personal communications page'.

example, a 'context help feature' DMSL component was added to the general work-list component to assist infrequent users of the application (e.g. legal advisor). For the network designer role, a set of interactor components providing access to document management facilities was added.

To support the account manager role it was necessary to provide an end-to-end overview of the quotation process and instances thereof. Hence, a group of components associated with PIF activity objects were used to provide a 'management' interface to the co-ordination service. These components enabled navigation through task structures in a basic textual hyperlink style and a more advanced graphical style with the look of IDEF0 process modelling diagrams [22].

6.4 *Demonstrator 1 — exploratory prototyping of user interface requirements for different team members*

The Web-based MCS implementation described in section 5 was first used to build a prototypical support environment for the customer quotation process. Within the MCS prototype, each role within the network design process was assigned a DMSL interpreter. By default, each of these interpreters was initially loaded with the general-purpose DMSL components described earlier. A library of customised DMSL components was then loaded into a further interpreter to be used for 'on-the-fly' prototyping purposes.

The demonstration started with a walkthrough showing user interaction with work-lists, work-items and information objects in the 'general' interaction style. Customisation of the user interface for the account manager role was then demonstrated (it was assumed that the account manager does not use the system frequently, and requires a simpler interface with a help feature). In the demonstration using the prototyping feature of the MCS implementation, a DMSL component providing a help feature is located and shown to the account manager, who then suggests some further stylistic changes. This DMSL component is then copied into the interaction model for the account manager role as a replacement for his current work-list interface and the model is then dynamically reloaded into the live interpreter. The account manager role then accesses their work-list again and uses the new interface. This component selection, customisation and update process takes less than 5 minutes, demonstrating the use of the MCS implementation in supporting rapid participatory prototyping of collaborative systems.

6.5 *Demonstrator 2 — business process evolution*

The ability of the MCS prototype to evolve over time was demonstrated using several sets of process, information and interaction models. Changes between the model sets

were used to simulate several types of evolutionary change in the business process and its supporting software. Several types of change were considered, e.g. change in general look and feel for the support system, change in role-specific user interface requirements, change in the user-interface requirements of individual users, change in the business process model, or a change in the information model. The above changes were demonstrated using two instances of the appropriate model(s). For example, in demonstrating change in the business process model, a new task was added into the customer quotation process. The existing PIF model for the process was copied and the extra task added, creating a new PIF model. Navigation through the existing quotation process was shown using generic DMSL components for interaction with PIF models expressed in IDEF0. The updated PIF model was then loaded into the MCS implementation via the PIF parser. The quotation process was then navigated again, using the existing IDEF diagram interface components, but showing the updated business process.

7. Evaluation

Evaluation was undertaken with research engineers studying aspects of supporting enterprise-wide business processes. Firstly, a set of slides to position the model-based approach and describe the construction of the DMSL component library were presented to the group. This was then followed by a walkthrough demonstration. Feedback on the perceived utility of the approach to BT was then gathered through a set of structured interviews, the results of which are summarised below.

Several strengths of the approach were noted:

- supporting the rapid exploration of design ideas with end-users was seen as the key advantage,
- the ability to dynamically integrate access to core services was seen as beneficial in contributing towards a 'value-added middleware layer' within dynamic enterprise environments,
- the flexible integration of collaboration, co-ordination and information management services was viewed as important,
- the capability to support a broad range of user interface delivery mechanisms was seen as advantageous, enabling deployment of new interaction styles (such as 3-D), while also managing existing baselines.

Some areas of weakness were also noted, however:

- the need for skilled (hence, expensive) software engineers to operate the MCS implementation was noted — use of graphical component construction tools was suggested as a solution to this problem (perhaps of a similar nature to JavaBean composition tools),

- issues of 'information abstraction' were not adequately addressed (e.g. the PIF model developed for the case study mixed process meta-descriptive information with process instance information),
- a problem of Web interface consistency within the MCS implementation was noted, indicating the need for a more sophisticated approach to session control.

8. Conclusions and future work

This paper has presented a model-based approach to the construction of integrated collaborative applications. The approach enables dynamic integration of core co-ordination, collaboration and information management services through reusable and customisable user-interface components. Such an approach is particularly appropriate to supporting the dynamic and heterogeneous characteristics of 'virtual teams'.

A prototype was implemented using Web/CORBA infrastructure and evaluated within BT. Although limited in scope, the results of the evaluation were encouraging and demonstrated both the feasibility and utility of the approach. The ability to readily customise the user interface at run-time enables collaborative systems to be prototyped in close to real time with end users, enhancing software usability through participatory design. The approach also has the potential to improve the evolutionary qualities of software systems to support virtual teams. Vanilla support could be provided very rapidly based on generic models, then gradually adapted to the requirements of the specific project and team. However, this latter topic requires further research investigation. Similarly, the approach opens the way for future investigation of personal working environments that combine a high degree of individualisation with task- and process-specific support. One can imagine a user interface that displays the appropriate information and tools when a user switches to a different project and task context, while retaining personal look and feel. This would require models of both user and process. The ability to display a role-centric view of the process would also be valuable for a virtual team worker.

Of course, the HCI community has been studying models for the construction of user interfaces for many years. For example, the task knowledge structure (TKS) models of Johnson and Johnson [23] are highly synergistic with the approach described here. They are concerned with practical techniques to model the cognitive intent of the user and mapping this into interface models. This is important because it could provide an underlying theory for the construction of DMSL components and, importantly, a sound methodology for their composition within an application context. This is the main direction for future work, in conjunction with more extensive evaluation and validation studies.

The research on the DMSL component library was undertaken over three years ago and there is now a need to update the implementation to reflect recent technology developments. For example, it would now be appropriate to utilise technologies such as extensible markup language (XML) and style sheets in the construction of DMSL components. The ability to appropriately integrate 3-D user interface technology with conventional Web content is increasingly relevant and there is further research needed to understand the role of hybrid 2-D/3-D graphical interfaces for supporting virtual teams. Finally, it is planned to investigate how this approach may be progressed through emerging enterprise-level component technologies such as Enterprise JavaBeans and the forthcoming CORBA 3.0 component model (CCM).

Acknowledgements

The DiMe architecture and DMSL language were devised by David Morris and Gyuri Lajos within the Centre for Virtual Working Environments at Leeds University. Christine Leigh, Richard Drew, Thorsten Blaise and Diane Willows also contributed to its development. The PIF parser used within the MCS prototype was written by Simon Thompson.

References

- 1 Goldman S L, Nagel R and Preiss K: 'Agile competitors and virtual organizations', Van Nostrand Reinhold (1994).
- 2 Swaby M A: 'A model-based approach to construction of integrated Internet CSCW systems', PhD thesis, School of Computer Studies, University of Leeds (1998).
- 3 National Industrial Information Infrastructure Protocol Consortium (NIIPC): 'NIIP Reference Architecture: Concepts and Guidelines', Technical Report NTR95-01 (January 1995).
- 4 Almasi G et al: 'Functional specifications for collaboration services', Technical Report CERC-TR-TM-94-002, Concurrent Engineering Research Center, West Virginia University (April 1994).
- 5 Pastor E and Jager J: 'Architectural framework for CSCW', in: 'Co-operation among organisations: the potential of computer supported cooperative work', ESPRIT Research Reports, Project 5660 (1993).
- 6 Sheth A: 'From contemporary workflow process automation to adaptive and dynamic work activity coordination and collaboration', Technical Report, Large Scale Distributed Information Systems Lab, University of Georgia (1997).
- 7 Petrie C (Ed): 'Enterprise integration modeling: proceedings of the first international conference', MIT Press (1992).

- 8 Object Management Group (OMG): 'The Common Object Request Broker: Architecture and Specification', OMG Document 97-09-01, Revision 2.1 (August 1997).
- 9 Morris D T, Lajos G, Dew P M, Drew R S and Willows D: 'DiMe: An object oriented scripting language for the automatic creation of virtual environments', in Proceedings of Eurographics, UK Chapter Conference (April 1997).
- 10 Dew P M, Leigh C M, Drew S, Morris D T and Curson J M: 'Collaborative working systems to support user interaction within a virtual science park', Information Services and Use, 15, pp 213—228 (1995).
- 11 Object Management Group (OMG): 'CORBA services: Common Object Services Specification', OMG Document 97-12-03 (November 1997).
- 12 Merle P, Gransart C and Gei J: 'CorbaWeb: A WWW and CORBA worlds integration', in Proceedings of the 2nd COOTS Workshop on Distributed Object Computing on the Internet, Toronto (June 1996).
- 13 Wernecke J: 'The inventor mentor: programming object oriented 3D graphics with Open Inventor', Addison Wesley (1994).
- 14 Parr T: 'Language Translation using PCCTS and C++', Automata Publishing Company (1996).
- 15 Hollingworth D: 'The workflow reference model', Workflow Management Coalition, Document TC00-1003, Issue 1.1 (November 1994).
- 16 Lee J, Gruninger M, Jin Y, Malone T, Tate A and Yost G: 'The PIF process interchange format and framework', Technical Report, PIF Working Group, University of Hawaii, Version 1.1 (May 1996).
- 17 ISO/IEC 9594-I, X.500 — The Directory, International Standard Specification (1992).
- 18 Lee J and Malone T: 'Partially shared views: a scheme for communicating amongst groups that use different type hierarchies', ACM Transactions on Office Information Systems, 8, No 1, pp 1—26 (1990).
- 19 World Wide Web Consortium: 'Document Object Model (DOM) Level 1 Specification', W3C Recommendation REC-DOM-Level-1-19981001, Version 1.0. (October 1998).
- 20 Pesce M: 'VRML: browsing and building cyberspace', New Riders (1995).
- 21 Jennings N R, Faratin P, Johnson M J, Norman T J, O'Brien P D and Wiegand M E: 'Agent-based business process management', International Journal of Cooperative Information Systems, 5, Nos 2 and 3, pp 105—130 (1996).
- 22 NIST: 'Integration definition for function modeling (IDEF0)', FIPS Publication 183 (December 1993).

- 23 Johnson P, Johnson H and Wilson S: 'Rapid prototyping of user interfaces driven by task models', in Carroll J M (Ed): 'Scenario-based design for human-computer interaction', Wiley (1995).



Mike Swaby obtained a BSc in computer science and software engineering from the University of Birmingham in 1994. He completed his PhD in model-based software architectures at Leeds University in 1998. His research was supported by an EPSRC Industrial CASE award through a collaborative project between the Centre for virtual working environments (CVWE) at Leeds University and the agent-enhanced workflow group at BT Laboratories. He currently works as a freelance contractor, specialising in CORBA software.



Professor Peter M Dew's research interests are in the area of multimedia architectures and systems to support virtual communities and knowledge management across the Internet. This builds upon his experience in the construction of virtual working systems, for which the first instance is the Leeds Virtual Science Park.

His current research is directed towards the second generation of virtual working systems using a model-based approach and, at a deeper level, understanding sharing architectures. His other research interests include parallel architectures and VR CAD. He is Professor of Computer Science, Director of the Centre for Virtual Working Systems and Technical Director of Virtual Working Systems Ltd.



Paul Kearney is a Senior Research Fellow working in the Intelligent Business Systems Research (IBSR) Group at BT Laboratories, Adastral Park. He has a BSc in Mathematical Physics (Liverpool University) and PhD in theoretical particle physics (Durham University). He spent nearly 10 years working for British Aerospace (Military Aircraft), firstly in aerodynamics, then progressing through graphics programming and then expert systems to R&D in on-board IKBS (tactical decision aids, etc) and software engineering. Joining Sharp Laboratories of Europe on 1990, he led a small research group applying agent concepts to personal electronics. He moved to BT in 1997. A particular research interest is in how complex collective behaviour patterns arise through the interaction of autonomous agents. His current work in the IBSR group pursues this interest in the context of providing support for business processes in a changing environment. He is BT's representative in PLANET, the European Network of Excellence in AI Planning and Scheduling, heading the technical co-ordination unit on Workflow Management, and on the Eurescom Project, MESSAGE (Methodology for Engineering Systems of Software Agents).